

# 实 验 报 告

|                 |                           |        |      |               |            |
|-----------------|---------------------------|--------|------|---------------|------------|
| 课程名称            | 软件测试                      |        |      |               |            |
| 实验项目名称          | 软件测试试验报告 3                |        |      |               |            |
| 实验时间<br>(日期及节次) | 第 16 周 3 - 4 节            |        |      |               |            |
| 专业              | 软件工程                      | 学生所在学院 |      | 计算机与大数据       |            |
| 年级              | 2024 级                    | 学号     |      | 20245109      |            |
| 姓名              | 刘健畅                       | 指导教师   |      | 徐为            |            |
| 实验室名称           | 4 号楼 - 413 [计算机 1 机房分室 1] |        |      |               |            |
| 实验成绩            | 预习情况                      | 操作技术   | 实验报告 | 附加：综合<br>创新能力 | 实验<br>综合成绩 |
|                 |                           |        |      |               |            |
| 教师签字            |                           |        |      |               |            |

黑龙江大学教务处

## 一. 实验目的与要求

1. 掌握黑盒测试中的边界值分析法。
2. 能够在等价类划分的基础上，针对输入域的边界设计高效的测试用例。
3. 掌握因果图法中输入条件的制约关系，并能结合具体程序进行分析。
4. 实施边界测试，分析并总结被测程序在边界处理上的缺陷与输入域测试结论。

## 二. 实验环境与工具

- 操作系统：Windows 11
- 开发语言及环境：Python, Vscode

## 三. 被测程序分析

被测程序 ConvertRMB.py 用于将阿拉伯数字金额转换为人民币大写金额。

隐含的输入域边界如下：

1. 整数部分长度边界：程序通过 GROUP\_CHARS = ["元", "万", "亿"] 划分单位。  
最多支持 3 个组（每组 4 位），即整数最大有效支持长度为 12 位。
2. 小数部分长度边界：规定小数部分长度必须为 2。如果不含小数点，系统默认补 00。
3. 数值边界：如 0.00、含连续零的数字（1001）、角为 0 分不为 0 的情况。

## 四. 等价类划分表

| 输入条件    | 有效等价类       | 编号 | 无效等价类   | 编号 |
|---------|-------------|----|---------|----|
| 整数部分长度  | 1 位-12 位    | 1  | 大于 12 位 | 2  |
|         |             |    | 0 位     | 3  |
| 小数部分长度  | 2 位         | 4  | 大于 2 位  | 6  |
|         | 0 位         | 5  | 1 位     | 7  |
| 字符类型    | 纯数字或仅含一个小数点 | 8  | 包含字母    | 9  |
|         |             |    | 包含特殊字符  | 10 |
|         |             |    | 包含空格    | 11 |
|         |             |    | 包含汉字    | 12 |
| 小数点出现次数 | 1 次         | 13 | 大于 1 次  | 15 |
|         | 0 次         | 14 |         |    |

## 五. 测试用例设计与执行结果

| 用例编号 | 输入数据                 | 覆盖边界/规则 | 预期输出                                            | 实际输出                                                            | 测试结果 |
|------|----------------------|---------|-------------------------------------------------|-----------------------------------------------------------------|------|
| 01   | .50                  | 整数长度下边界 | 人民币伍角                                           | 人民币伍角                                                           | Pass |
| 02   | .05                  | 整数长度下边界 | 人民币伍分                                           | 人民币零伍分                                                          | Fail |
| 03   | 1.00                 | 整数长度下边界 | 人民币壹元整                                          | 人民币壹元整                                                          | Pass |
| 04   | 9999999<br>99999.99  | 整数长度上边界 | 人民币<br>玖仟玖佰玖拾玖亿<br>玖仟玖佰玖拾玖万<br>玖仟玖佰玖拾玖元<br>玖角玖分 | 人民币<br>玖仟玖佰玖拾玖亿<br>玖仟玖佰玖拾玖万<br>玖仟玖佰玖拾玖元<br>玖角玖分                 | Pass |
| 05   | 1000000<br>000000.00 | 整数长度上边界 | 整数长度超过 12                                       | IndexError: list index out of range                             | Fail |
| 06   | 123                  | 小数长度下边界 | 人民币壹佰贰拾叁元整                                      | 人民币壹佰贰拾叁元整                                                      | Pass |
| 07   | 123.4                | 小数长度下边界 | 格式错误... 小数部分长度不为 2...!                          | 格式错误... 小数部分长度不为 2...!<br>IndexError: string index out of range | Fail |
| 08   | 123.45               | 小数标准长度  | 人民币壹佰贰拾叁元肆角伍分                                   | 人民币壹佰贰拾叁元肆角伍分                                                   | Pass |
| 09   | 123.456              | 小数长度上边界 | 格式错误... 小数部分长度不为 2...!                          | 格式错误... 小数部分长度不为 2...!<br>人民币壹佰贰拾叁元肆角伍分                         | Fail |

|    |              |                |                                       |                                                                  |      |
|----|--------------|----------------|---------------------------------------|------------------------------------------------------------------|------|
| 10 | 12a.00       | 包含字母           | 格式错误                                  | ValueError:<br>invalid literal<br>for int() with<br>base 10: 'a' | Fail |
| 11 | 12@.00       | 包含特殊<br>字符     | 格式错误                                  | ValueError:<br>invalid literal<br>for int() with<br>base 10: '@' | Fail |
| 12 | 12 3.00      | 包含空格           | 格式错误                                  | ValueError:<br>invalid literal<br>for int() with<br>base 10: ' ' | Fail |
| 13 | 12 元.00      | 包含汉字           | 格式错误                                  | ValueError:<br>invalid literal<br>for int() with<br>base 10: '元' | Fail |
| 14 | 123456789.12 | 全 汉 字 覆<br>盖   | 人民币壹亿贰仟叁<br>佰肆拾伍万陆仟柒<br>佰捌拾玖元壹角贰<br>分 | 人民币壹亿贰仟叁<br>佰肆拾伍万陆仟柒<br>佰捌拾玖元壹角贰<br>分                            | Pass |
| 15 | 1409.50      | 中间单零           | 人民币壹仟肆佰零<br>玖元伍角                      | 人民币壹仟肆佰零<br>玖元伍角                                                 | Pass |
| 16 | 6007.14      | 连续零            | 人民币陆仟零柒元<br>壹角肆分                      | 人民币陆仟零柒元<br>壹角肆分                                                 | Pass |
| 17 | 1680.32      | 元位零            | 人民币壹仟陆佰捌<br>拾元叁角贰分                    | 人民币壹仟陆佰捌<br>拾元叁角贰分                                               | Pass |
| 18 | 16409.02     | 角 为 零 分<br>不为零 | 人民币壹万陆仟肆<br>佰零玖元零贰分                   | 人民币壹万陆仟肆<br>佰零玖元零贰分                                              | Pass |

|    |              |               |                    |                                                             |      |
|----|--------------|---------------|--------------------|-------------------------------------------------------------|------|
| 19 | 100001000.00 | 亿组有数，<br>万组全零 | 人民币壹亿零壹仟<br>元整     | 人民币壹亿零壹仟<br>元整                                              | Pass |
| 20 | 0.00         | 全零            | 人民币整               | 人民币整                                                        | Pass |
| 21 | 10000.00     | 一万整           | 人民币壹万元整            | 人民币壹万元整                                                     | Pass |
| 22 | 100000000.00 | 一亿整           | 人民币壹亿元整            | 人民币壹亿元整                                                     | Pass |
| 23 | 107000.53    | 跨组零           | 人民币壹拾万零柒<br>仟元伍角叁分 | 人民币壹拾万零柒<br>仟元伍角叁分                                          | Pass |
| 24 | 123.4.5      | 多个小数<br>点     | 格式错误               | ValueError: too<br>many values to<br>unpack (expected<br>2) | Fail |

## 六. 因果图法中的输入域约束条件分析

### 6.1 异或约束(E)

定义：多个输入原因中最多只有一个可能成立，不能同时成立。

小数点数量：

原因 A（输入金额包含 1 个小数点）、原因 B（输入金额不包含小数点）、原因 C（输入金额包含 2 个及以上小数点）。

### 6.2 要求约束(R)

定义：当原因 A 出现时，原因 B 必须出现。

“零”的补充规则：

当原因 A（需要输出“零 X 分”，如：零伍分）成立时，原因 B（整数部分存在且不为“0”）必须成立。

### 6.3 唯一约束(O)

定义：多个条件中，必须有且仅有一个条件成立。

对于任意一个指定的数字位，原因 A（属于元组）、原因 B（属于万组）、原因 C（属于亿组）具有唯一约束（O）。一个数字所在的 unit\_id 在同一时刻只能唯一对应数组 GROUP\_CHARS 中的某一个下标。

## 七. 缺陷分析与测试结论

### 7.1 缺陷发现

在本次测试执行中，共发现四处严重的健壮性与逻辑缺陷：

1. 多小数点引发的崩溃：

当输入包含 2 个或以上小数点（如 123.4.5）时，由于源码采用 `int_part, frac_part = money.split(".")` 进行强行解包，切分出的元素数量大于预期的 2 个，导致 Python 抛出 `ValueError: too many values to unpack`。

2. 逻辑错误缺陷：

当测试整数长度极小边界（长度为 0，如输入 .05）时，程序 `split` 得到的 `int_part` 为空字符串 `""`。在后续角分处理时，代码 `if int_part != "0":` 误将空字符串判定为“包含非零整数”（即满足了上述的要求约束 R），从而错误地输出了 人民币零伍分（预期应为 人民币伍分）。

3. 极大边界导致的数组越界：

当整数部分长度达到上边界越界值（13 位及以上，如万亿级别）时，程序计算出的 `unit_id` 为 3。而代码中单位数组 `GROUP_CHARS = ["元", "万", "亿"]` 最大下标仅为 2。执行拼接时直接引发数组越界崩溃。

4. 拦截失效与非法字符崩溃：

当小数部分长度不等于 2，或者输入包含字母/符号时，程序虽然通过 `print` 提示了“格式错误”，但缺少 `return` 语句中断控制流。程序继续向下强行执行字符串切片 `frac_part[1]` 或类型转换 `int()`，引发二次崩溃。

### 7.2 修改缺陷

1. 增加 `money.count(".") > 1` 的校验；所有异常 `print` 提示后，必须追加 `return` 中断程序。

2. 将判断条件修改为 `if int_part != "0" and int_part != ""`。

3. 增加 `len(int_part) > 12` 的拦截，并在转换前使用替换和 `isdigit()` 保证字符合法。

### 7.3 测试结论

本次实验综合运用等价类划分、边界值分析与因果图分析法，共计执行了二十余个测试用例。结果表明该程序算法正常，健壮性较差。

## 八. 附录

学号: 20245109

姓名: 刘健畅

```
NUM_CHARS = ["零", "壹", "贰", "叁", "肆", "伍", "陆", "柒", "捌", "玖"]
# 人民币每 4 位一组, 1000 是 1000 元. 1,0000 就是 1 万元
GROUP_CHARS = ["元", "万", "亿"]
# 每组中不同位置的读法, 1000 中, 1 用仟; 10,0000 中, 10 用拾(对应数组中下标 1 的拾, 从右向左
# 数, 1 在第 1 个位置, 下标从 0 开始)
POS_CHARS = ["", "拾", "佰", "仟"]

def ConvertRMB(money):
    if "." in money:
        int_part, frac_part = money.split(".")
    else:
        int_part = money
        frac_part = "00"

    if (len(frac_part) != 2):
        print("格式错误...小数部分长度不为 2...!\n")

    print(f"{int_part}.{frac_part}\n")

    res = ""

    # 整数部分
    if int_part != "0":
        zero_count = 0 # 记录连续出现的 0 的个数
        n = len(int_part) # 整数长度, 用于计算当前遍历的数字处于哪个组, 在组中的哪个位置

        # 遍历每一位数
        for i in range(n):
            digit = int(int_part[i])
            pos = n - i - 1 # 该数字距离末尾的位置, 用于计算组号与组中的位置
            unit_id = pos // 4 # 整数除法, 向下取整, 得到该数字在第几组
            sub_id = pos % 4 # 该数字在组内的位置

            if digit > 0:
                if (zero_count > 0):
                    res += "零" # 当前数字不是 0, 而且前面有 0, 加上零
                res += NUM_CHARS[digit] + POS_CHARS[sub_id] # 该数字变成大写人民币, 加上
            # 他组内的读法

            zero_count = 0
        else:
            zero_count += 1

    # 到了这个组的最后面, 看看要不要加这个组的单位
```

```

        if sub_id == 0:
            group_start = max(0, i - 3) # 向前找这个组的开始下标, 如果不满 4 位就是 0
            group_val = int(int_part[group_start:i + 1]) # 提取从该组的数字(i + 1)
是因为右面是开区间

            if group_val > 0 or unit_id == 0: # 如果该组没有为 0 的数字, 或者到了最后一组(需要加上元)
                res += GROUP_CHARS[unit_id] # 加上对应组的单位

    if frac_part == "00":
        res += "整"
    else:
        jiao = int(frac_part[0])
        fen = int(frac_part[1])

        if jiao > 0:
            res += NUM_CHARS[jiao] + "角"
        elif jiao == 0 and fen > 0: # 角是 0, 分不是 0
            if int_part != "0":
                res += "零" # 整数部分不为零, 在元后面补零

        if fen > 0:
            res += NUM_CHARS[fen] + "分"

    print(f"人民币{res}\n")

```